

ALBERI E ALBERI BINARI

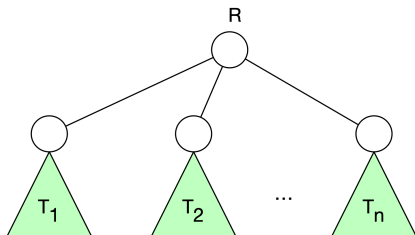
PIETRO DI LENA

DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
UNIVERSITÀ DI BOLOGNA

ALGORITMI E STRUTTURE DI DATI
ANNO ACCADEMICO 2024/2025



- Un **Albero** è una struttura dati non-lineare ad **albero gerarchico**
- Definizione di struttura dati Albero:
 - Un insieme di **nodi** (or vertici)
 - Un insieme di **archi** che connettono nodi
 - Esiste **un solo percorso** per andare da un nodo all'altro
- Un Albero è **ordinato** se i figli di ogni nodo sono ordinati
 - Possiamo identificare il primo figlio, il secondo figlio, ...
- Un albero è **radicato** se uno dei suoi nodi è identificato come **radice**

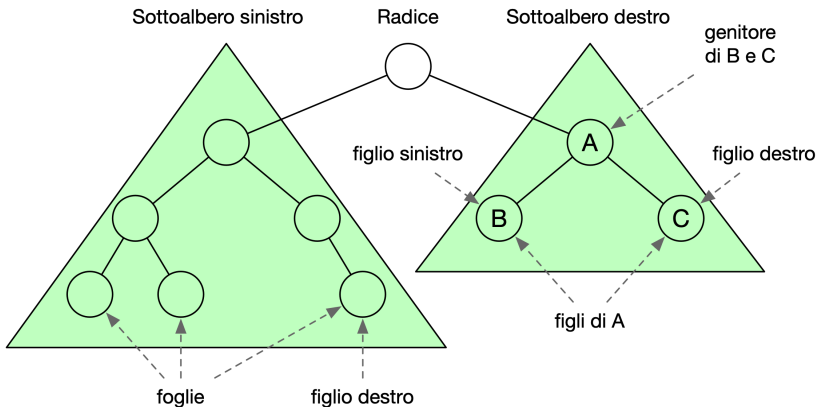


Definizione ricorsiva di Albero radicato:

- Insieme vuoto di nodi oppure
- Una radice R e zero o più alberi disgiunti (sotto-alberi) le cui radici sono connesse ad R

ALBERI BINARI

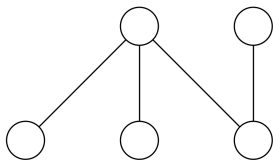
- Un **Albero Binario** è un Albero in cui ogni nodo ha **al massimo due figli**



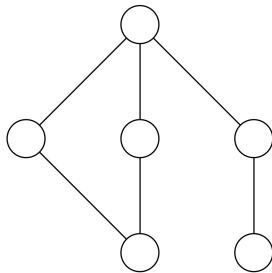
- Un Albero Binario è un **Albero ordinato**
 - Ogni nodo può avere un figlio **sinistro** e/o un figlio **destro**
 - Un nodo può avere figlio destro ma non sinistro (e al contrario)

QUALI SONO ALBERI (BINARI)?

A



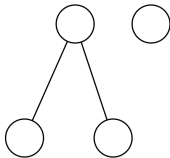
B



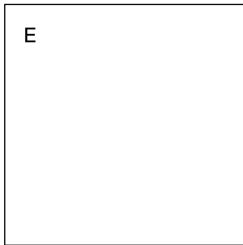
C



D

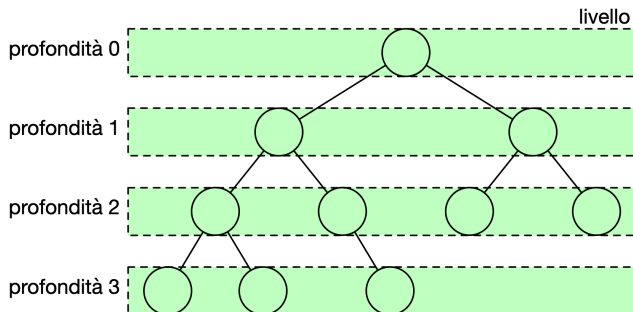


E



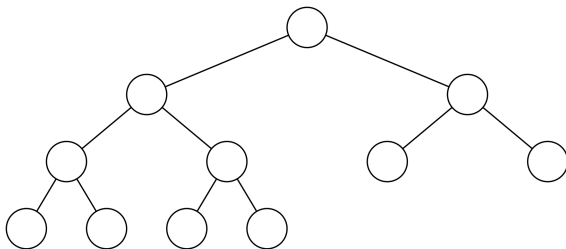
ALCUNE DEFINIZIONI

- La **profondità** di un nodo u è la lunghezza del percorso (unico) che va dalla radice al nodo u (numero di archi)
- Un **livello** è l'insieme di tutti i nodi alla stessa profondità
- L'**altezza** di un Albero è la sua massima profondità
 - Per convenzione, un albero vuoto ha altezza -1
- Il **grado** di un nodo è il numero dei suoi figli

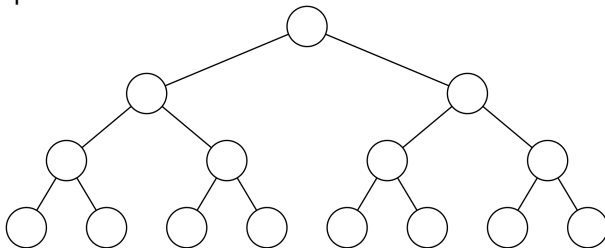


ALBERI BINARI SPECIALI

- Un albero binario è **completo** se ogni nodo intermedio ha due figli



- Un albero binario è **perfetto** se è completo e se tutte le foglie sono alla stessa profondità



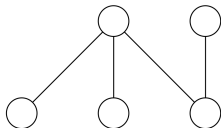
PROPRIETÀ FONDAMENTALE DI UN ALBERO

Teorema

Ogni Albero non-vuoto con n nodi ha esattamente $n - 1$ archi

(Dimostrazione per induzione)

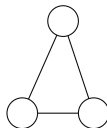
- Possiamo usare tale proprietà per dimostrare che una struttura dati **non** è un Albero
- Il Teorema non funziona nella direzione opposta



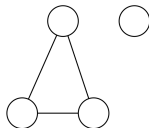
5 nodi
4 archi



1 nodo
0 archi



3 nodi
3 archi



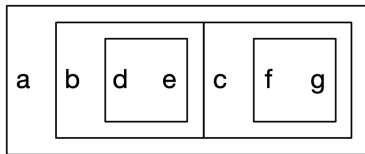
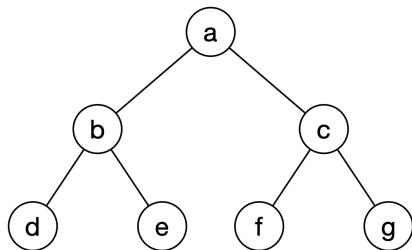
4 nodi
3 archi

ALGORITMI DI VISITA SU ALBERI

- **Algoritmo di visita** (o anche *algoritmo di ricerca*) su Albero
 - Algoritmo per *visitare* tutti i nodi di una struttura dati Albero
- **Visita in profondità** o Depth-First Search (DFS)
 - La ricerca va in profondità il più possibile prima di visitare il nodo successivo nello stesso livello
 - Esistono tre varianti: pre-ordine, post-ordine, in-ordine
- **Visita in ampiezza** o Breadth-First Search (BFS)
 - La ricerca viene eseguita livello per livello

VISITA IN PROFONDITÀ: PRE-ORDINE (PRE-ORDER)

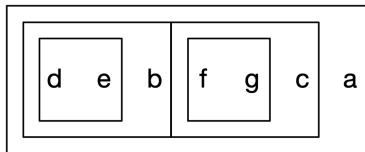
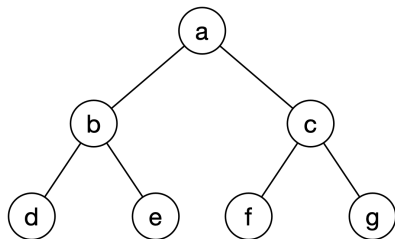
```
1: function PREORDER(NODE  $T$ )  
2:   if  $T \neq \text{NIL}$  then  
3:     VISIT( $T$ )  
4:     PREORDER( $T.\text{left}$ )  
5:     PREORDER( $T.\text{right}$ )
```



- NODE indica una struttura nodo di un albero
- Assumiamo che VISIT abbia costo $O(1)$
- Costo (ottimo, pessimo, medio): $\Theta(n)$ (n = numero di nodi)

VISITA IN PROFONDITÀ: POST-ORDINE (POST-ORDER)

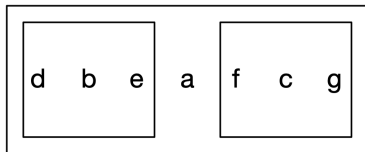
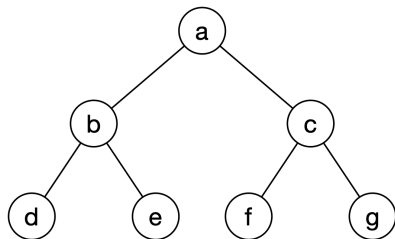
```
1: function POSTORDER(NODE  $T$ )  
2:   if  $T \neq \text{NIL}$  then  
3:     POSTORDER( $T.\text{left}$ )  
4:     POSTORDER( $T.\text{right}$ )  
5:     VISIT( $T$ )
```



Costo (ottimo, pessimo, medio): $\Theta(n)$ (n = numero di nodi)

VISITA IN PROFONDITÀ: IN-ORDINE (IN-ORDER)

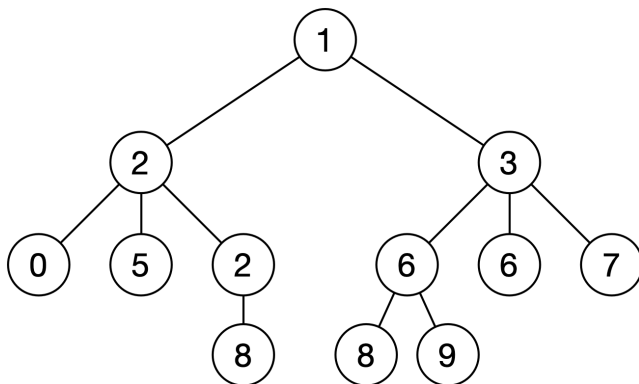
```
1: function INORDER(NODE  $T$ )  
2:   if  $T \neq \text{NIL}$  then  
3:     INORDER( $T.\text{left}$ )  
4:     VISIT( $T$ )  
5:     INORDER( $T.\text{right}$ )
```



Costo (ottimo, pessimo, medio): $\Theta(n)$ (n = numero di nodi)

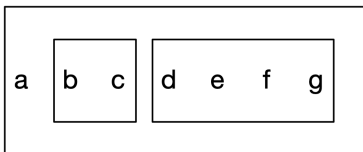
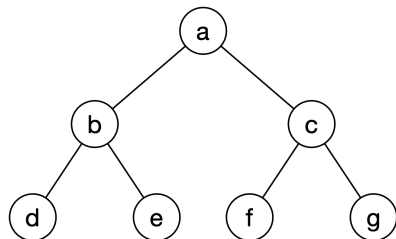
VISITA IN PROFONDITÀ SU ALBERI NON-BINARI

- Gli algoritmi di visita possono essere generalizzati ad Alberi non Binari
- Solo la visita in-ordine richiede specifiche aggiuntive
 - Es: visitiamo i primi k figli, poi nodo corrente, poi i figli rimanenti
- Qual è l'ordine di visita (pre,post,in) nel seguente Albero?



VISITA IN AMPIEZZA (BFS)

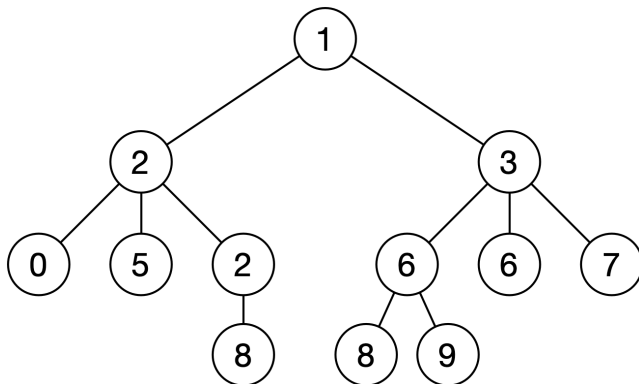
```
1: function BFS(TREE  $T$ )  
2:    $Q = \text{QUEUE}()$   
3:   if  $T.\text{root} \neq \text{NIL}$  then  
4:      $\text{ENQUEUE}(Q, T.\text{root})$   
5:   while  $Q.\text{size} \neq 0$  do  
6:      $x = \text{DEQUEUE}(Q)$   
7:      $\text{VISIT}(x)$   
8:     if  $x.\text{left} \neq \text{NIL}$  then  
9:        $\text{ENQUEUE}(Q, x.\text{left})$   
10:    if  $x.\text{right} \neq \text{NIL}$  then  
11:       $\text{ENQUEUE}(Q, x.\text{right})$ 
```



- N.B. Usiamo una Coda per imporre un ordine di visita per livello
- Costo (ottimo, pessimo, medio): $\Theta(n)$ (n = numero di nodi)

VISITA IN AMPIEZZA SU ALBERI NON-BINARI

- Anche la visita in ampiezza è generalizzabile ad Alberi non-binari
- Qual è l'ordine di visita in ampiezza dei nodi nel seguente Albero?



ESEMPIO: ALGORITMO SU ALBERO BINARIO

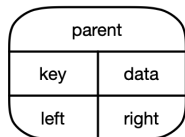
Scrivere un algoritmo per contare i nodi di un Albero Binario

```
1: function COUNTNODES(Node  $T$ )  $\rightarrow$  INT
2:   if  $T == NIL$  then
3:     return 0
4:   else
5:     return 1 + COUNTNODES( $T.left$ ) + COUNTNODES( $T.right$ )
```

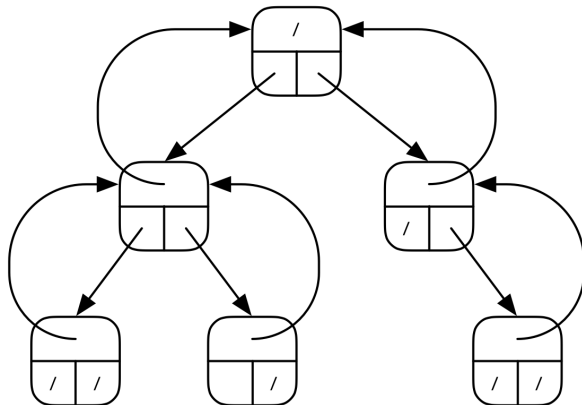
- Quale tipo di visita è implementato in COUNTNODES?
 - post-ordine (per valutare T bisogna prima valutare i suoi figli)
- Qual è il tempo di calcolo di COUNTNODES?
 - Costo (ottimo, pessimo, medio): $\Theta(n)$
- Come modificare l'Albero Binario in modo che COUNTNODES sia $O(1)$?
 - Aggiungiamo un campo $T.totnodes$ ad ogni nodo
- Qual è l'impatto di tale modifica sul costo di altre operazioni, ad esempio *aggiungi una nuova foglia*, su un Albero Binario?

IMPLEMENTAZIONE DI UN ALBERO BINARIO

Nodo



Albero Binario

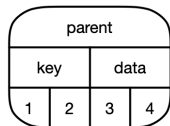


Implementazione con puntatori

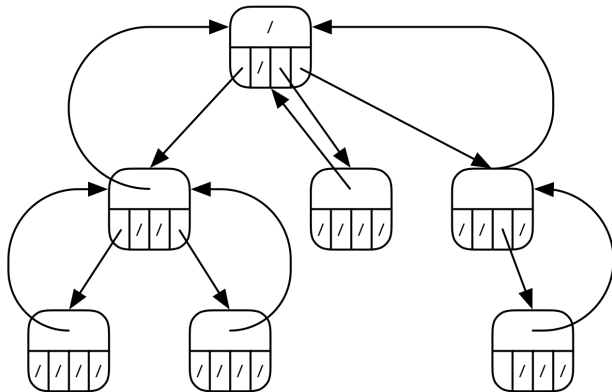
- left: puntatore al figlio sinistro
- right: puntatore al figlio destro

IMPLEMENTAZIONE DI UN ALBERO NON-BINARIO 1

Nodo



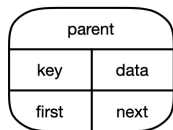
Albero



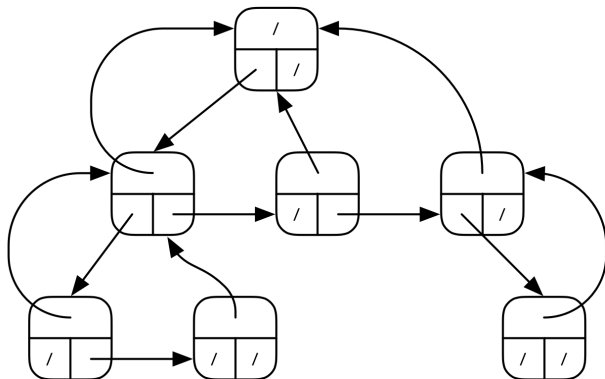
- Ogni nodo contiene un **array di puntatori** a k figli
- Il **numero massimo di k figli** è fisso
- Rischio di sprecare spazio se molti nodi hanno meno di k figli

IMPLEMENTAZIONE DI UN ALBERO NON-BINARIO 2

Nodo



Albero



- Ogni nodo ha un puntatore al **primo** (first) figlio
- Ogni nodo ha un puntatore al fratello **successivo** (next)
- La lista di figli è gestita con una Lista concatenata semplice

VISITE IN PROFONDITÀ SU ALBERO NON BINARIO

Visite per la rappresentazione primo-figlio, fratello-successivo

```
// Solo ricorsiva
1: function PREORDER(Node T)
2:   if T  $\neq$  NIL then
3:     VISIT(T)
4:     PREORDER(T.first)
5:     PREORDER(T.next)
```

```
// Solo ricorsiva
1: function POSTORDER(Node T)
2:   if T  $\neq$  NIL then
3:     POSTORDER(T.first)
4:     POSTORDER(T.next)
5:     VISIT(T)
```

```
// Mista ricorsiva/iterativa
1: function PREORDER(Node T)
2:   if T  $\neq$  NIL then
3:     VISIT(T)
4:     tmp = T.first
5:     while tmp  $\neq$  NIL do
6:       PREORDER(tmp)
7:       tmp = tmp.next
```

```
// Mista ricorsiva/iterativa
1: function POSTORDER(Node T)
2:   if T  $\neq$  NIL then
3:     tmp = T.first
4:     while tmp  $\neq$  NIL do
5:       POSTORDER(tmp)
6:       tmp = tmp.next
7:     VISIT(T)
```

Costo (ottimo, pessimo, medio) in tutti i casi: $\Theta(n)$ (n = numero di nodi)

VISITA IN AMPIEZZA SU ALBERO NON BINARIO

Visita BFS per la rappresentazione primo-figlio, fratello-successivo

```
1: function BFS(TREE T)
2:   LET Q BE A new QUEUE
3:   if T.root  $\neq$  NIL then
4:     ENQUEUE(Q, T.root)
5:   while Q.size  $\neq$  0 do
6:     x = DEQUEUE(Q)
7:     VISIT(x)
8:     tmp = x.first
9:     while tmp  $\neq$  NIL do
10:      ENQUEUE(Q, tmp)
11:      tmp = tmp.next
```

Costo (ottimo, pessimo, medio): $\Theta(n)$ (n = numero di nodi)